

Experiment 6

Aim: Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE (Voting Ballot).

Theory:

Relevance of `require` Statements in Solidity Programs

In Solidity, the `require` statement acts as a guard condition within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code.

If the condition inside `require` is not satisfied, the function execution stops immediately and all state changes made during that transaction are reverted to their original state. This prevents invalid transactions from corrupting blockchain data.

For example, in a Voting Smart Contract, `require` can be used to check:

- Whether the person calling the function has the right to vote.
- Whether a voter has already voted.
- Whether the function caller is the chairperson before granting voting rights.

Thus, `require` statements provide security, correctness, and reliability in smart contracts. They can also include error messages that make debugging and contract interaction easier.

Keywords: mapping, storage, and memory

1. mapping:

A mapping is a special data structure in Solidity that links keys to values, similar to a hash table.

Syntax:

```
mapping(keyType => valueType)
```

Example:

```
mapping(address => Voter) public voters;
```

Here, each Ethereum address is mapped to a `Voter` structure. Mappings are useful in contracts such as Ballot, where voter information needs to be associated with individual addresses.

Mappings do not have a length property and cannot be directly iterated over. They are therefore efficient for lookups but limited for enumeration.

2. storage:

Storage refers to the permanent memory of the smart contract stored on the Ethereum blockchain.

Variables declared at the contract level are stored in storage by default. Data stored in storage persists across transactions unless explicitly modified.

However, writing to blockchain storage consumes gas, making it more expensive. For example, voter information stored in a mapping remains available throughout the contract's lifecycle.

3. memory:

Memory is temporary storage used only during the lifetime of a function call. Once the function execution ends, the data stored in memory is discarded.

Memory is mainly used for:

- Temporary variables
- Function arguments
- Temporary computations
- String manipulation

Memory is cheaper than storage in terms of gas cost. Therefore, developers need to balance the use of storage and memory for efficiency and cost-effectiveness.

Why **bytes32** instead of **string**?

In earlier implementations of the Ballot contract, **bytes32** was used for proposal names because of its efficiency and gas optimization.

- **bytes32** → fixed-size type that stores exactly 32 bytes. It provides simpler storage, faster comparisons, and lower gas costs. However, proposal names are limited to 32 characters.
- **string** → dynamically sized type that can store text of variable length. It is more user-friendly but requires more complex handling inside the Ethereum Virtual Machine, which can increase gas usage.

Modern implementations may use **string** to make the system more user-friendly. Tools such as Web3 Type Converter can be used to switch between the two types.

In summary, **bytes32** is useful when performance and gas efficiency are priorities, while **string** is preferred for readability and ease of use.

Code:

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.20;

contract VotingBallot {

    struct Candidate {

        string name;

        uint256 voteCount;

    }

    Candidate[] public candidates;

    mapping(address => bool) public hasVoted;

    address public owner;

    event CandidateAdded(

        uint256 indexed candidateId,

        string name

    );

    event VoteCast(

        address indexed voter,

        uint256 indexed candidateId

    );

    modifier onlyOwner() {

        require(
```

```
msg.sender == owner,

        "Only owner can perform this action"

    );

    _;

}

constructor() {

    owner = msg.sender;

}

function addCandidate(

    string memory _name

) public onlyOwner {

    require(

        bytes(_name).length > 0,

        "Candidate name cannot be empty"

    );

    candidates.push(

        Candidate({

            name: _name,

            voteCount: 0

        })

    );
```

```

emit CandidateAdded(
    candidates.length - 1,
    _name
);
}

function vote(
    uint256 _candidateId
) public {

    require(
        !hasVoted[msg.sender],
        "You have already voted"
    );

    require(
        _candidateId < candidates.length,
        "Invalid candidate"
    );

    hasVoted[msg.sender] = true;
    candidates[_candidateId].voteCount++;

    emit VoteCast(
        msg.sender,

```

```

        _candidateId
    );
}

function getCandidateCount()
    public
    view
    returns (uint256)
{
    return candidates.length;
}

function getCandidate(
    uint256 _candidateId
)
    public
    view
    returns (
        string memory,
        uint256
    )
{
    require(
        _candidateId < candidates.length,
        "Invalid candidate"
    );
}

```

```

Candidate memory candidate =
    candidates[_candidateId];

return (
    candidate.name,
    candidate.voteCount
);
}

```

```

function getWinner()
    public
    view
    returns (
        string memory winnerName,
        uint256 winnerVotes
    )
{
    require(
        candidates.length > 0,
        "No candidates available"
    );
}

```

```

uint256 winningIndex = 0;

for (
    uint256 i = 1;
    i < candidates.length;
    i++
) {
    if (
        candidates[i].voteCount >
        candidates[winningIndex].voteCount
    ) {
        winningIndex = i;
    }
}

winnerName =
    candidates[winningIndex].name;

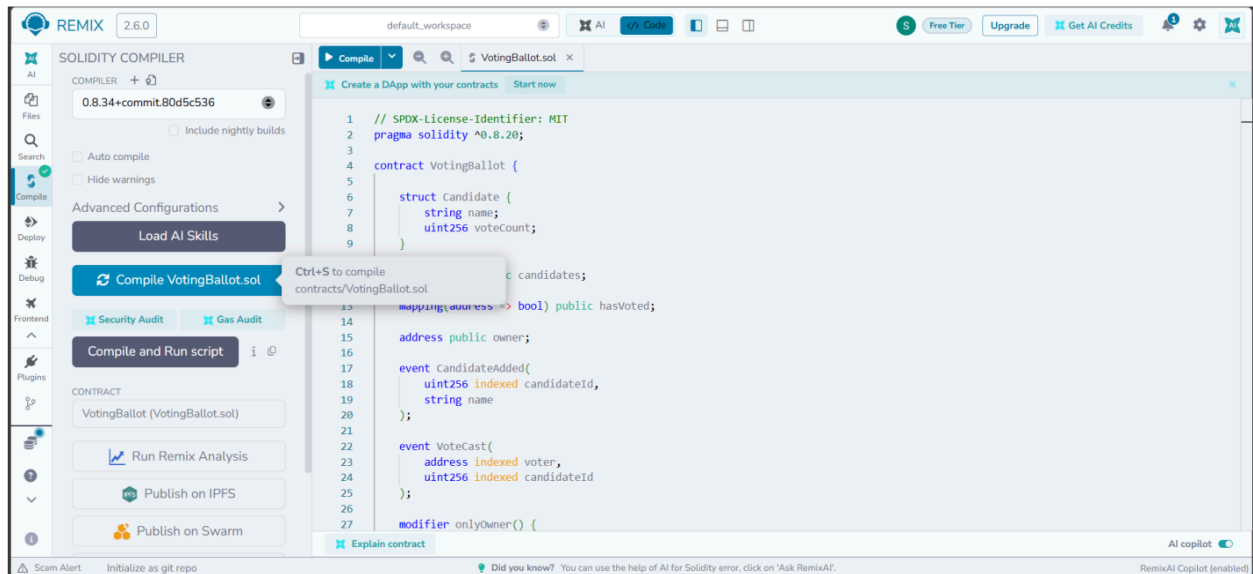
winnerVotes =
    candidates[winningIndex].voteCount;
}
}

```

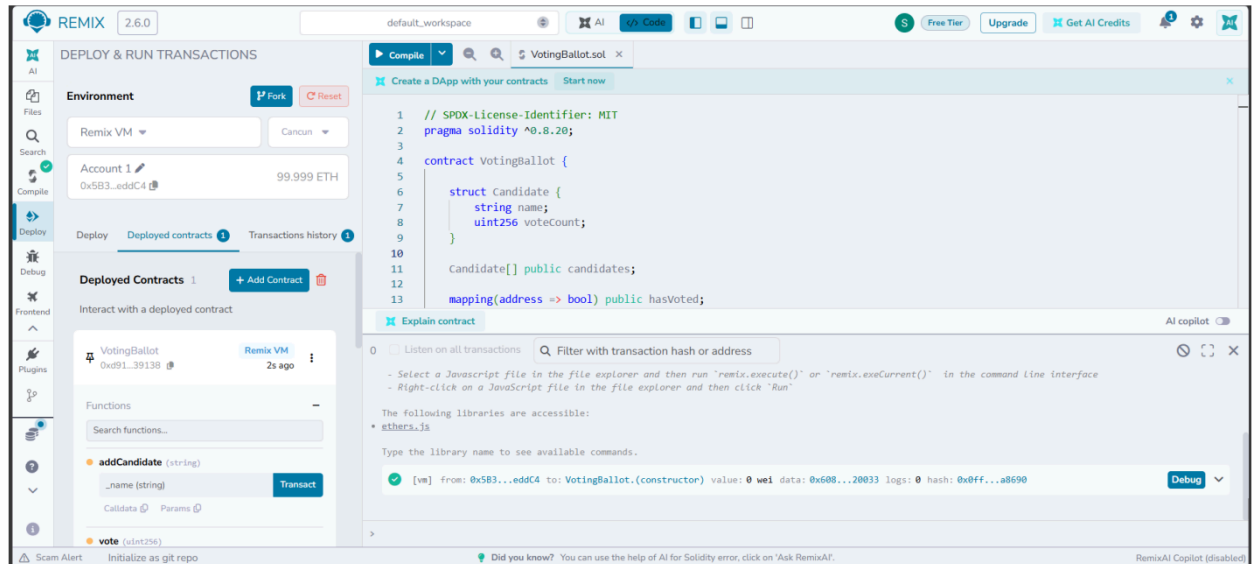
The contract contains candidate management, voting, prevention of double voting, candidate-count retrieval, candidate details retrieval, and winner determination.

Output:

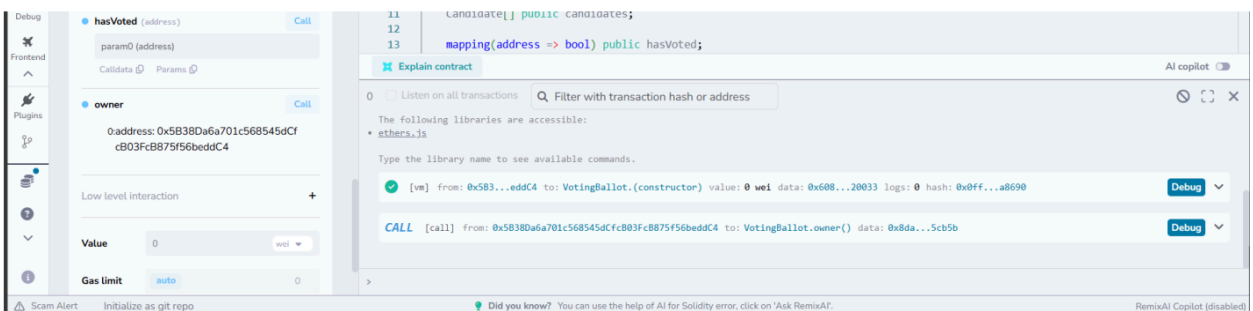
Contract Compiled:



Contract Deployed:



Owner of contract:



Added Alice as candidate:

The screenshot shows the Remix IDE interface. On the left, the 'Functions' panel lists `addCandidate` (string), `vote` (uint256), and `candidates` (uint256). The `addCandidate` function is selected, and its parameters are set to `Alice` and `param0` (uint256). The `Transact` button is visible. The main editor shows the Solidity code for the `VotingBallot` contract, including the `addCandidate` function. The right sidebar shows the 'Explain contract' panel with a list of transactions. The first transaction is a `CALL` from `0x58380a6a701c568545dcfc803fc8875f56beddC4` to `VotingBallot.owner()` with data `0x8da...5cb5b`. The second transaction is a `CALL` from `0x58380a6a701c568545dcfc803fc8875f56beddC4` to `VotingBallot.addCandidate(string)` with data `0x462...00000` and logs `1 hash: 0xf5a...6070d`.

Added 2nd Candidate Bob:

The screenshot shows the Remix IDE interface. The `addCandidate` function is selected, and its parameters are set to `Bob` and `param0` (uint256). The `Transact` button is visible. The main editor shows the Solidity code for the `VotingBallot` contract. The right sidebar shows the 'Explain contract' panel with a list of transactions. The first transaction is a `CALL` from `0x58380a6a701c568545dcfc803fc8875f56beddC4` to `VotingBallot.owner()` with data `0x8da...5cb5b`. The second transaction is a `CALL` from `0x58380a6a701c568545dcfc803fc8875f56beddC4` to `VotingBallot.addCandidate(string)` with data `0x462...00000` and logs `1 hash: 0xf5a...6070d`. The third transaction is a `CALL` from `0x58380a6a701c568545dcfc803fc8875f56beddC4` to `VotingBallot.addCandidate(string)` with data `0x462...00000` and logs `1 hash: 0xbd3...6d51a`.

Added 3rd Candidate Charlie:

The screenshot shows the Remix IDE interface. The `addCandidate` function is selected, and its parameters are set to `Charlie` and `param0` (uint256). The `Transact` button is visible. The main editor shows the Solidity code for the `VotingBallot` contract. The right sidebar shows the 'Explain contract' panel with a list of transactions. The first transaction is a `CALL` from `0x58380a6a701c568545dcfc803fc8875f56beddC4` to `VotingBallot.owner()` with data `0x8da...5cb5b`. The second transaction is a `CALL` from `0x58380a6a701c568545dcfc803fc8875f56beddC4` to `VotingBallot.addCandidate(string)` with data `0x462...00000` and logs `1 hash: 0xf5a...6070d`. The third transaction is a `CALL` from `0x58380a6a701c568545dcfc803fc8875f56beddC4` to `VotingBallot.addCandidate(string)` with data `0x462...00000` and logs `1 hash: 0xbd3...6d51a`. The fourth transaction is a `CALL` from `0x58380a6a701c568545dcfc803fc8875f56beddC4` to `VotingBallot.addCandidate(string)` with data `0x462...00000` and logs `1 hash: 0xccc...c7087`.

Calling `getCandidateCount`:

The screenshot shows the Remix IDE interface. The `getCandidateCount` function is selected, and its parameters are set to `0` (uint256). The `Call` button is visible. The main editor shows the Solidity code for the `VotingBallot` contract. The right sidebar shows the 'Explain contract' panel with a list of transactions. The first transaction is a `CALL` from `0x58380a6a701c568545dcfc803fc8875f56beddC4` to `VotingBallot.addCandidate(string)` with data `0x462...00000` and logs `1 hash: 0xbd3...6d51a`. The second transaction is a `CALL` from `0x58380a6a701c568545dcfc803fc8875f56beddC4` to `VotingBallot.addCandidate(string)` with data `0x462...00000` and logs `1 hash: 0xccc...c7087`. The third transaction is a `CALL` from `0x58380a6a701c568545dcfc803fc8875f56beddC4` to `VotingBallot.addCandidate(string)` with data `0x462...00000` and logs `1 hash: 0xccc...c7087`. The fourth transaction is a `CALL` from `0x58380a6a701c568545dcfc803fc8875f56beddC4` to `VotingBallot.getCandidateCount()` with data `0x30a...56347`.

The output is **3**, confirming that three candidates were successfully added.

Calling **getCandidate** function on Alice:

The screenshot shows the Remix IDE interface. On the left, the 'Deployed contracts' panel lists the 'candidates' contract with parameters 'param0 (uint256)' and 'Calldata'. Below it, the 'getCandidate' function is shown with parameters 'O:string: Alice' and '1:uint256: 0'. On the right, the 'Transactions history' panel shows a list of transactions. The first transaction is a 'CALL' from '0x58380a6a701c568545dCfc803fc8875f56beddC4' to 'VotingBallot.getCandidateCount()' with data '0x30a...56347'. The second transaction is a 'CALL' from '0x58380a6a701c568545dCfc803fc8875f56beddC4' to 'VotingBallot.getCandidate(uint256)' with data '0x35b...00000'. The third transaction is a 'CALL' from '0x58380a6a701c568545dCfc803fc8875f56beddC4' to 'VotingBallot.addCandidate(string)' with data '0xd91...39138' and value '0 wei'.

Alice initially had **0** votes.

Voting for Alice using Account 2:

The screenshot shows the Remix IDE interface. On the left, the 'Deployed contracts' panel lists the 'vote' contract with parameters 'Charlie' and 'Transact'. Below it, the 'getCandidate' function is shown with parameters '0' and 'Calldata'. On the right, the 'Transactions history' panel shows a list of transactions. The first transaction is a 'CALL' from '0x58380a6a701c568545dCfc803fc8875f56beddC4' to 'VotingBallot.getCandidateCount()' with data '0x30a...56347'. The second transaction is a 'CALL' from '0x58380a6a701c568545dCfc803fc8875f56beddC4' to 'VotingBallot.getCandidate(uint256)' with data '0x35b...00000'. The third transaction is a 'CALL' from '0x58380a6a701c568545dCfc803fc8875f56beddC4' to 'VotingBallot.vote(uint256)' with data '0xd91...39138' and value '0 wei'.

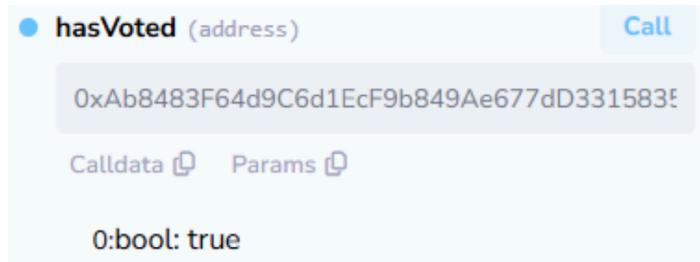
Account 2 successfully voted for Alice.

Verify that the vote is given to Alice:

The screenshot shows the Remix IDE interface. On the left, the 'Deployed contracts' panel lists the 'getCandidate' function with parameters '0' and 'Calldata'. Below it, the 'O:string: Alice' and '1:uint256: 1' are shown. On the right, the 'Transactions history' panel shows a list of transactions. The first transaction is a 'CALL' from '0x58380a6a701c568545dCfc803fc8875f56beddC4' to 'VotingBallot.getCandidateCount()' with data '0x30a...56347'. The second transaction is a 'CALL' from '0x58380a6a701c568545dCfc803fc8875f56beddC4' to 'VotingBallot.getCandidate(uint256)' with data '0x35b...00000'. The third transaction is a 'CALL' from '0x58380a6a701c568545dCfc803fc8875f56beddC4' to 'VotingBallot.vote(uint256)' with data '0xd91...39138' and value '0 wei'.

Alice's vote count became **1**.

Verify Account 2 has voted:



Preventing double voting:



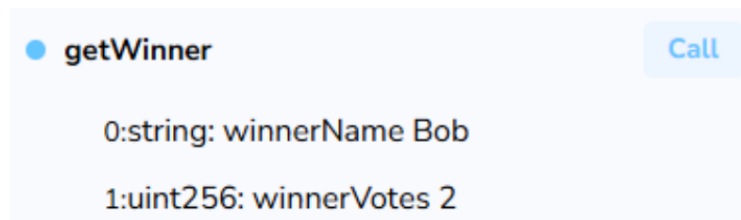
Double voting was successfully prevented by the contract.

Using 3rd and 4th account to vote for Bob (1):





Finding the winner using **getWinner**:



Conclusion:

In this experiment, a Voting/Ballot smart contract was deployed using Solidity on the Remix IDE. The concepts of **require** statements, mapping, and data-location specifiers such as storage and memory were explored to understand their role in ensuring security, efficiency, and correctness in smart contracts.

The difference between using **bytes32** and **string** for proposal names was also studied, highlighting the trade-off between gas efficiency and readability. Overall, the experiment provided practical insights into the design and deployment of voting contracts on the blockchain.